Birmingham Purchase Card Data Workflow

Patrick Gilliland

Udacity

AI Mastery Capstone: AI Programming Foundations

Udacity mentor

October 4, 2026

Birmingham Purchase Card Data Workflow

Overview

I built a reusable data workflow in one Jupyter notebook, `data_workflow.ipynb`, that loads, cleans, summarises and charts Birmingham City Council's purchase card transactions for 2024. The question was where the council's card spend went, and how far the raw record can be trusted to answer that. The dataset is Purchase Card Transactions, published by Birmingham City Council (2026) on the Birmingham City Observatory under the Open Government Licence v3.0 (https://cityobservatory.birmingham.gov.uk/explore/dataset/purchase-card-transactions/information/).

Dataset Description

Each row is one transaction on a council purchase card. The council publishes the file monthly, and its page explains that the Code of Recommended Practice for Local Authorities on Data Transparency encourages councils to publish these transactions (Birmingham City Council, 2026). The 2024 file holds 42,560 rows and 10 columns, covering 1 January to 31 December 2024. I focused on five variables: transaction date, amount billed in pounds, original currency, merchant name, and the directorate that made the purchase. The council masked card numbers except for the last four digits, so I did not analyse them. The only blanks were 562 in the tax description, a column I did not use.

Workflow Description

The notebook follows the lifecycle Danchev (2022) describes: begin with a research question, then select the data, preprocess it and describe it. After a Setup cell that imports the libraries, its steps are:

- **Ingestion.** The file loads with a plain `pd.read_csv` call, and I checked its shape, types and blanks.
- **Cleaning.** Four functions: standardise the column names and types; normalise merchant names; harmonise directorate names; flag refunds and use the amount billed in pounds, since some purchases were in other currencies. Each takes a DataFrame and returns a new one, so the loaded data is never changed and each step can be rerun alone.
- **Screening (end of Cleaning).** A fifth function, `flag_unexplained_payments`, marks payments the data cannot explain. One merchant is flagged and set aside from the primary analysis, with its rows kept.
- **Exploratory data analysis (EDA).** One function, `summarise_spend`, returns net total (after refunds, including the recorded tax), transaction count, median transaction and refund share for any grouping. I ran it by directorate, month and merchant, and by channel on the spend of Amazon and the suppliers whose names I joined: Amazon's buying route, or the channel a supplier's name shows, such as its stores, its filling stations or online. A second function, `prime_charges_for_review`, lists the Amazon Prime charges outside Schools that a finance team might check.
- **Visualizations.** Three charts drawn from those summaries (Figures 1 to 3).
- **Summary.** My interpretation of what the data shows and where it falls short.

Key Decisions and Assumptions

Cleaning Choices

The raw file had four problems, one for each cleaning function. Column names had capitals and spaces, and dates arrived as text, so I standardised both. The file was already tidy in Wickham's (2014) sense, one row per transaction and one column per variable, so the work here was the other cleaning tasks he lists, such as "parsing dates and numbers" and "matching similar but not identical values" (p. 21). Merchant names are the card terminal's own text, cut at 25 characters and often carrying order or store numbers, so one supplier appears under many names. Normalising them took the distinct count from 14,972 to 4,581. I treated Amazon's retail, marketplace and business names as one supplier, Amazon, and applied one joining rule to every other supplier whose names, combined, would reach the top 15 merchants. A name joins its supplier when the supplier's own name is a whole word at its start, such as "tesco pfs 4231" and "tesco gift card", or is the name in its web address, such as "www.argos.co.uk". A few names stay apart because the supplier's name is neither a whole word at their start nor the name in their web address, such as the MFG filling station "pfs @mfg kings norton" and EDF's "edfenergy.com". One Amazon name, "amzn digital", stays apart too, because it matches none of the patterns that join Amazon's names. The names left apart total £9,483.56 and change no position in the top 15 if added to their suppliers, so I left them apart as immaterial to the question. I kept the channel each purchase came through in its own column, so the split stays visible. Directorate names changed during the year, which the publisher warns can happen after internal changes (Birmingham City Council, 2026). I assumed that "Adult Social Care (eip)" is part of Adult Social Care, because it runs alongside the plain name from April, so I rolled it up and went from 12 values to 11. The file does not say what "eip" stands for. Finally, 282

purchases were in other currencies, so I used the amount billed in pounds; summing original amounts would have counted Korean won as pounds.

No step changed the loaded data or the downloaded file, and in the merchant and directorate steps, I kept the raw value in its own column. Wilson et al. (2017) argue that raw data must be kept so an analysis can be rerun from start to finish, and that every processing step should be recorded. Following Danchev (2022), who notes that notebooks let an analyst record "dead ends" and "transparent choices of parameter selection" (p. 3), each cleaning step in the notebook also lists the rules I considered and rejected, such as joining "argos ltd" with "argosy toys".

Screening

Before the analysis, I screened the cleaned data for payments the file cannot explain. One merchant, "bham council phone pyt", took £1,052,247.67 from 291 transactions, 18.06% of the published net total, and 287 of them were booked to City Housing. By its name, it is the council taking a payment from itself, but no column says what for. Van den Broeck et al. (2005) describe data cleaning as screening, diagnosis and treatment. Values that come from a process the study did not plan for may be left out, but this "becomes an a posteriori exclusion criterion and the data points should be reported as 'excluded from analysis'" (Van den Broeck et al., 2005, p. 969). I followed that. The rows are kept and flagged, not deleted, and set aside from the primary analysis. Stoudt et al. (2021) make the same point: if we remove data points, "we should know which data points we removed", why, and be able to explain both when we share the work (p. 8). The notebook reports the published net total, £5,825,200.86, beside the total analysed, £4,772,953.19. In practice, I would ask the council's finance team what these payments are before drawing any conclusion about City Housing. Smaller council-related merchants that the

data does explain, such as Clean Air Zone charges, stay in. So does the merchant named just "bcc", which reads as Birmingham City Council: £940.00 from 3 transactions, unexplained but small, and the notebook records it.

Assumptions

Refunds are kept as negatives, so all totals are net of refunds (1,422 refunds, 3.34% of transactions). Amounts include the recorded tax. Uncertain directorate renames were left apart, which may split one service across two names, one used in January and February and the other from March. No limit is published in the data, so from the values of the council's phone payments I used an assumed £10,000 limit for these payments. It is an assumption about these payments only, not a council-wide card limit. Results explains why it matters.

What EDA Focused on

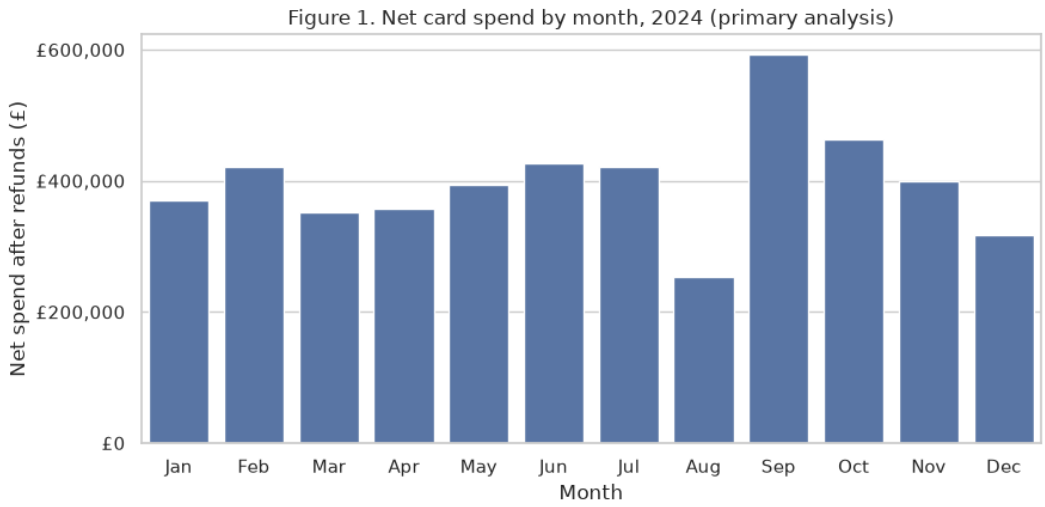
I focused on where the money goes (directorate and merchant) and when (month), because those answer the question directly. I also split the spend of Amazon and the joined suppliers by channel, and noted the Amazon Prime charges, listing those outside Schools that a finance team might check.

What Each Plot Was Designed to Show

Figure 1 shows net spend by month, to see how spend moves through the year. Figure 2 shows net spend by directorate, to show which services use the cards. Figure 3 shows the top 15 merchants after cleaning, to show who is paid.

Figure 1

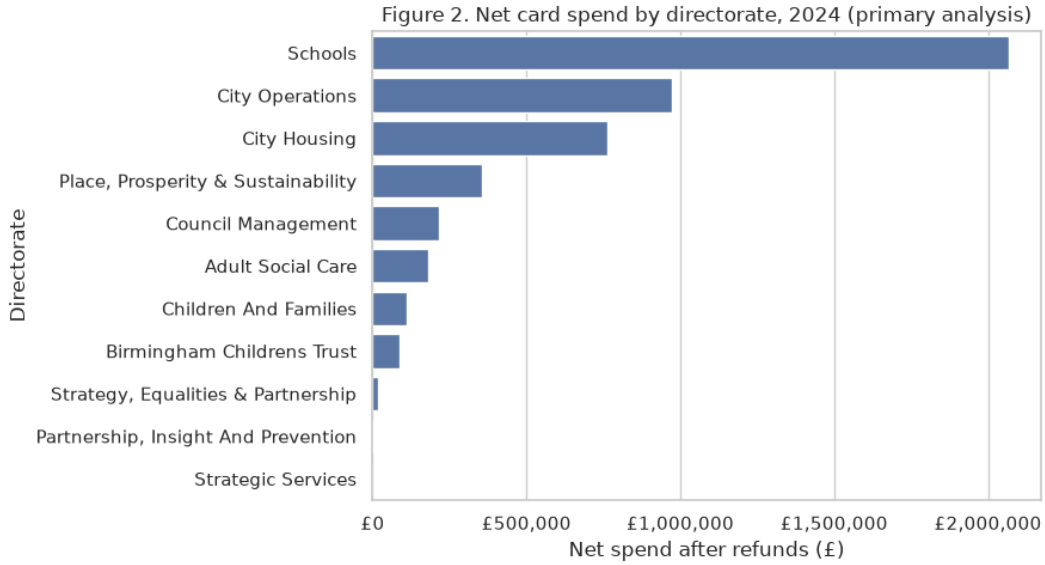
Net Card Spend by Month (Primary Analysis)



Note. Purchase card transactions for 2024, from Birmingham City Council (2026).

Figure 2

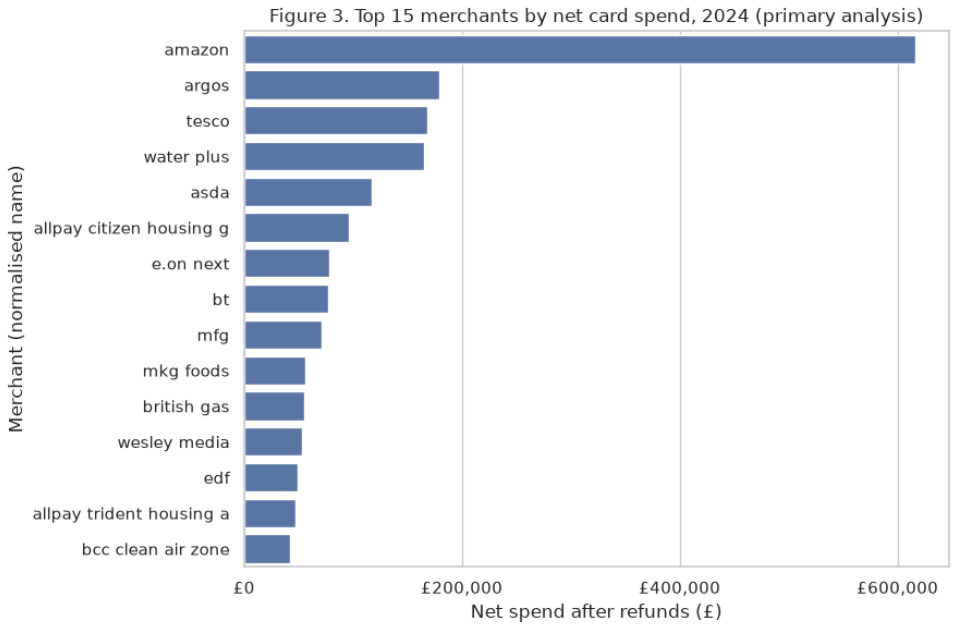
Net Card Spend by Directorate (Primary Analysis)



Note. Purchase card transactions for 2024, from Birmingham City Council (2026).

Figure 3

Top 15 Merchants by Net Card Spend (Primary Analysis)



Note. Purchase card transactions for 2024, from Birmingham City Council (2026).

Results and Interpretation

The most important result came from the screening, not the charts. Nearly a fifth of the published net card spend, 18.06%, is one unexplained merchant that by its name is the council paying itself. Setting it aside changes the answer to the question. With it, City Housing looks like the second largest user of the cards at £1,816,193.02. Without it, City Housing falls to third at £764,969.36, from 2,564 transactions. Van den Broeck et al. (2005) recommend "separate analyses with and without" suspect values (p. 969), and this with-and-without comparison is shown in the notebook.

My working reading, from experience of council and enterprise finance systems, is that City Housing is paying amounts owed to the council itself, such as rent or council tax on temporary accommodation, through the council's own card payment line. That would make them

internal transfers, not spend with suppliers. Internal does not mean neutral for budgets, though: whether a payment uses up a directorate's budget or only moves money between budgets depends on who pays and who receives, and the file cannot tell. Before relying on any of this, I would ask the council four things. What is the phone payment line used for? Which accounts and budgets receive these payments? Are they charged to City Housing's budget, or do they net off within it? And what limit applies to these payments? No limit is published in the data. From the values, I used an assumed £10,000 limit for these payments: the largest is £9,910.23 and 5 of the 291 payments are £9,000 or more, so the payments closest to it could be candidates for review. That alone does not show split transactions. I am not saying that happened, but payments close to that limit are one of the first things I would check.

In the primary analysis, net card spend was £4,772,953.19. Schools had the largest share, £2,068,978.14, and by far the most transactions, 25,273, with a median of £35.77 (Figure 2). City Operations came second at £971,469.55. Monthly spend peaked in September (£593,575.10) and was lowest in August (£253,333.19), when school transactions dropped to 381, against 2,787 in September (Figure 1 and the month table in the notebook's Cleaning section). The largest supplier was Amazon, £616,469.34 from 13,163 transactions across its retail, marketplace and business channels, with Argos next at £179,343.34 and Tesco third at £167,886.40 (Figure 3). Asda stood out for refunds: in its online channel, 33.37% of 1,807 transactions were refunds.

Responsible Practice (Bias and Data Quality)

Cleaning is where this dataset could most easily mislead. A loose merchant rule would merge different suppliers and shift totals between them, so I applied only rules I could check against real names and kept the raw names. Merging directorates on a guess would move spend between services, so I merged only the one case the timing supports. Dropping refunds would

overstate spend, and dropping foreign purchases would understate it, so I kept both and used the amount billed in pounds. Setting aside the unexplained council payments is itself a choice that could mislead if hidden, so both totals are reported (Van den Broeck et al., 2005), and the rows stay in the data with their reason. The data also has a coverage bias. Purchase cards are one payment method of council spending, and part of this file appears to be the council paying itself, so it cannot describe the council's spending as a whole. Some Amazon buyer references appear to contain staff names. The cleaning collapses them, and the notebook never prints them.

The notebook also notes 256 Prime-labelled Amazon transactions, £6,286.18 net. Of these, 26 are Amazon Business Prime, by its name a business subscription, and 202 of the rest are in Schools, where a subscription is expected to be institutional. The notebook lists the other 28 Amazon Prime transactions, £252.05 net, by directorate, and I looked at them: most directorates have one or two charges, and City Housing has a charge in each month from March to December. Whether any needs action depends on approval records the file does not hold, so the list makes no finding against anyone.

Reproducibility

Someone else can rerun this work from the public repository, https://github.com/PaddyGilliland1/ai-programming-foundations-project, by following its README: with Python 3.12 or newer, create a virtual environment, run `pip install -r requirements.txt`, then open the notebook, restart the kernel and run all cells. The dataset sits beside the notebook, and `requirements.txt` lists every package and version. It was written with `pip freeze` from a fresh environment in which the notebook had run from top to bottom, then tested by rebuilding a second environment from the file alone. That follows Rule et al. (2019), who advise: "Always conduct your work in an environment created only from these

dependencies to ensure you do not add undocumented dependencies" (p. 4). Danchev (2022) explains that a requirements file listing packages and versions lets others rebuild the same environment, "enabling reproducibility and minimising possible errors due to package updates" (p. 4), and Wilson et al. (2017) recommend making dependencies explicit in the same way. Danchev also notes that a notebook "does not guarantee reproducibility" (p. 3) on its own, which is why the environment is pinned. I checked the code against the Python style guide with Black, set to the style guide's line length, and flake8, as Danchev's own resource does. The Git history shows the work: the setup commit on `main`, then a commit for each step on a `dev` branch, which is merged into `main` before submission. Wilson et al. (2017) recommend using a version control system, and name Git as the most widely discussed.

Sources and Citations

References

Birmingham City Council. (2026). *Purchase card transactions* [Data set]. Birmingham City Observatory. Retrieved September 25, 2026, from https://cityobservatory.birmingham.gov.uk/explore/dataset/purchase-card-transactions/information/

Danchev, V. (2022). Reproducible data science with Python: An open learning resource. *Journal of Open Source Education, 5*(56), Article 156. https://doi.org/10.21105/jose.00156

Rule, A., Birmingham, A., Zuniga, C., Altintas, I., Huang, S.-C., Knight, R., Moshiri, N., Nguyen, M. H., Rosenthal, S. B., Pérez, F., & Rose, P. W. (2019). Ten simple rules for writing and sharing computational analyses in Jupyter Notebooks [Editorial]. *PLOS Computational Biology, 15*(7), Article e1007007. https://doi.org/10.1371/journal.pcbi.1007007

Stoudt, S., Vásquez, V. N., & Martinez, C. C. (2021). Principles for data analysis workflows. *PLOS Computational Biology, 17*(3), Article e1008770. https://doi.org/10.1371/journal.pcbi.1008770

Van den Broeck, J., Argeseanu Cunningham, S., Eeckels, R., & Herbst, K. (2005). Data cleaning: Detecting, diagnosing, and editing data abnormalities. *PLoS Medicine, 2*(10), Article e267. https://doi.org/10.1371/journal.pmed.0020267

Wickham, H. (2014). Tidy data. *Journal of Statistical Software, 59*(10), 1-23. https://doi.org/10.18637/jss.v059.i10

Wilson, G., Bryan, J., Cranston, K., Kitzes, J., Nederbragt, L., & Teal, T. K. (2017). Good enough practices in scientific computing. *PLOS Computational Biology, 13*(6), Article e1005510. https://doi.org/10.1371/journal.pcbi.1005510